ADAPTING SMALL LANGUAGE MODELS TO ROMANIZED NEPALI: TOKENIZER FERTILITY, REFERENCE LIKELIHOOD AND GENERATION METRICS

A PREPRINT

Pujan Pant[1,*], **Sagar Newpane**[1], **Hari Sitaula**[2], **and Manish Sakhakarmy**[3,*]

[1]*Herald College Kathmandu, University of Wolverhampton, Naxal, Kathmandu 44600, Nepal*
[2]*Department of Mathematical Sciences, Montana Technological University, Butte, MT 59701, USA*
[3]*ASI Industrial, Billings, MT 59101, USA*
[*]*Correspondence: pantpujan017@gmail.com (P.P.); sakhakarmym@gmail.com (M.S.)*

ABSTRACT

Romanized Nepali, Nepali written in Latin script, is common in digital communication but has no standard spelling and few resources, and it is unclear how reliably small open language models can be adapted to it. We adapt three 2–3B base models from different families, Llama-3.2-3B, Qwen2.5-3B and Gemma-2-2B, with QLoRA on 9,000 instruction examples produced by deterministic transliteration of Nepali instruction data. Each model is trained with one shared recipe and three seeds and evaluated on 1,000 held-out examples with reference-overlap metrics, byte-normalized reference likelihood (bits per byte, BPB) and an output audit, against zero-shot baselines under two prompts. Fine-tuning raises chrF++ from 10–14 to 29.0–30.6 and lowers BPB from 2.23–2.41 to 0.78–0.84; most zero-shot outputs (54–92%) are English-like. Likelihood and overlap disagree on how far apart the models are. Llama-3.2-3B has the lowest BPB in every seed, 0.042 below Qwen2.5-3B (95% CI 0.039–0.046), yet the two cannot be separated on chrF++ (30.51 and 30.61) or BLEU, even with paired item-level tests. Gemma-2-2B, whose tokenizer splits Romanized Nepali words least (2.72 tokens per word), scores worst on every measure. Which model gains most over its zero-shot baseline depends on the zero-shot prompt. In a supplementary comparison with an identical tokenizer, Llama-3.1-8B lowers BPB by 0.053 (0.050–0.056) relative to Llama-3.2-3B, while chrF++ and BLEU rise by only 0.68 and 0.58 points. Lower fertility and higher reference likelihood therefore did not identify the model whose outputs were closest to the references.

Keywords Romanized Nepali · transliteration · small language models · parameter-efficient fine-tuning · QLoRA · tokenizer fertility · bits per byte · evaluation · low-resource languages

1 Introduction

Nepali is written in Devanagari, yet much of its everyday digital use is in Latin script. More than half of the comments in a Nepali social-media corpus were transliterated or mixed [31], and users of a Nepali sexual and reproductive health chatbot wrote 61.0% of their queries in Romanized Nepali, against 23.2% in Devanagari and 15.8% in English [46, Fig. 4A]. Romanized Nepali has no standard orthography: the same word can be spelled in several ways, and English words are often mixed in [5]. Language models also handle romanized input less well than the native script. In a real-world triage setting, one strong model's F1 on Romanized Nepali messages was 9.5 points below its F1 on Devanagari messages [23].

There are two broad responses. One converts the input: spelling normalization improved sentiment classification of Romanized Nepali reviews [5], and transliterating comments back to Devanagari helped multilingual classifiers [33]. The other adapts a model to read and write Romanized Nepali directly, which suits generation, where the output itself must be in the script that users type. Small open models of 2–3B parameters make this route practical, because parameter-efficient methods let them be adapted on a single 16 GB GPU [6], as in this study.

Choosing and evaluating such models is not straightforward. Candidate models differ at once in tokenizer, architecture, size and pretraining data. Tokenizer fertility, the number of subword tokens per word, is an appealing selection criterion, because text that is rare in a tokenizer's training data is split into many pieces [34, 42], but its relation to downstream performance is inconsistent across studies [1, 13, 43]. Per-token perplexity cannot be compared across models with different tokenizers [11]. A model's likelihood on reference text need not reflect the properties of the text it generates [28], and single training runs hide how much results vary with the random seed [8]. The closest prior benchmark for Romanized Nepali compared three 7–8B models with one run each and per-token perplexity [41], leaving open how small models compare and which measures can be trusted to rank them.

We study the adaptation of three 2–3B base models from different families, Llama-3.2-3B, Qwen2.5-3B and Gemma-2-2B, to a Romanized Nepali instruction task. The task is controlled: Romanized text was produced from Devanagari instruction data by deterministic, lowercased ITRANS transliteration, so every word has one spelling (Section 3). All models share one QLoRA recipe, a validation split for checkpoint selection and three training seeds. We measure tokenizer fertility, byte-normalized reference likelihood (bits per byte, BPB) and reference-overlap metrics, audit the generated outputs, and compare against zero-shot baselines under two prompt formats. A supplementary comparison with Llama-3.1-8B, whose tokenizer encodes our references identically to that of Llama-3.2-3B, adds a within-family contrast. We ask four questions:

- **RQ1.** How do three 2–3B base-model families adapt to Romanized Nepali under one parameter-efficient recipe, and how stable are the outcomes across training seeds?
- **RQ2.** Does tokenizer fertility on Romanized Nepali indicate which model adapts best?
- **RQ3.** Do reference likelihood and generation overlap agree on how models compare, after adaptation and within one model family when tokenization is identical?
- **RQ4.** How do base models behave on this task without adaptation, and how sensitive are zero-shot baselines, and gains measured from them, to the prompt?

The paper makes four contributions:

(1) A multi-seed evaluation of three 2–3B base-model families adapted to Romanized Nepali under one QLoRA recipe, reporting both training-seed and test-item uncertainty.

(2) Evidence that byte-normalized reference likelihood and generation overlap give different pictures of adapted models, including in a tokenizer-identical within-family comparison.

(3) A fertility analysis of four tokenizers on the benchmark references and, under three romanization conventions, on public Nepali text, in which the least-fragmenting tokenizer belonged to the lowest-scoring model and the order of tokenizers was the same under every convention.

(4) An analysis of zero-shot behaviour showing that the base models mostly answer in English and that the order of models by gain over zero-shot changes with the prompt.

We document the scripts, hashes and package versions behind every run (Appendix A). The benchmark is a deterministic transliteration of instruction data. It does not test robustness to the spelling variation of naturally typed Romanized Nepali, to mixed scripts or to code-switching, and all measures are automatic, so our conclusions concern adapting and evaluating models on this controlled task. Section 2 reviews related work, Sections 3 and 4 describe the data and experimental setup, Section 5 reports the results, and Sections 6–8 discuss them.

2 Background and Related Work

2.1 Romanized Nepali

Nepali is written in Devanagari, an abugida in which a consonant letter carries an inherent vowel unless it is marked otherwise. The script distinguishes some sounds that spoken Nepali does not: it has long and short *i* and *u*, but spoken Nepali has no vowel-length contrast [22]. Other distinctions are contrastive in speech, such as aspiration and the contrast between dental and retroflex stops [22]. Writers of Romanized Nepali have no agreed way of marking these distinctions in Latin script, so spellings vary across writers [5]. Nepali NLP in Devanagari includes sequence-labelling tools [39], tokenizer comparisons for language models [27] and continual pretraining of an 8B model on Nepali [10]. Work on Romanized Nepali has mostly addressed classification: offensive-language detection in social media, where transliterated text was common [31]; sentiment analysis after spelling normalization [5]; and sentiment analysis after transliteration back to Devanagari with NepaliXlit [33].

2.2 Romanized and Transliterated Text in Language Models

Romanized text in South Asian languages is common online, and language models handle it unevenly. In real-world triage messages in six Indian languages including Nepali, F1 on romanized text was 5–12 points lower than on native script [23]. For Sinhala, a benchmark of language models found far higher perplexity on romanized than on native-script text, measured per token across different tokenizers [40]. Romanization can also be used deliberately: RomanSetu uses romanized text as an interface to English-centric models, reducing token fertility by a factor of 2–4 relative to native scripts and matching native-script performance on a range of tasks [17]. In this journal, Baruah et al. [3] characterized how Romanized Assamese is written in social media and built machine transliteration for it. These studies concern natural romanized text or romanization as a tool. The adaptation of small generative models to produce romanized text, and how such adaptation should be evaluated, has received less attention.

2.3 Tokenization and Model Performance

Subword tokenizers built with byte-pair encoding or SentencePiece [25, 44] split text that is rare in their training data into more pieces. Fertility, the average number of tokens per word, is the usual measure of this [42]. Tokenized length differs by up to 15 times across languages for the same content [34], and for encoder models a dedicated monolingual tokenizer can matter about as much as the amount of pretraining data [42]. Whether fertility predicts downstream performance is less clear. In controlled pretraining, fertility and parity were not always predictive of downstream results [1]; text compression tracked performance, especially for generation tasks and smaller models [13]; and a tokenizer that minimized token count did not yield better models [43]. For Turkish, a study in this journal found a morphological tokenizer competitive with, but not consistently better than, statistical tokenizers [48], and for Devanagari Nepali, SentencePiece tokenizers performed better than byte-level BPE on understanding tasks [27]. In language transfer, extending a model's vocabulary to shorten target-language text was not beneficial at training scales of tens of billions of tokens [50]. These studies vary the tokenizer under controlled pretraining or transfer. Our question is narrower: whether the fertility of existing tokenizers helps choose among pretrained models to adapt.

2.4 Parameter-Efficient Adaptation for Low-Resource Languages

Low-rank adaptation (LoRA) trains small low-rank updates to frozen weights [16]. QLoRA applies it to a 4-bit quantized base model, which reduces memory enough to fine-tune large models on a single GPU [6], and rank-stabilized LoRA (rsLoRA) scales the update by $\alpha/\sqrt{r}$ rather than α/r, which prevents learning from slowing at higher ranks [20]. These methods are widely used to adapt open models to low-resource languages. LoRA fine-tuning of Gemma on a translated Alpaca dataset for Marathi lowered automatic scores while manual judgments favoured the tuned model [21]; parameter-efficient fine-tuning of 3–7B models for Bengali hate-speech classification found Llama-3.2-3B best [19]; continual pretraining with 4-bit QLoRA taught Llama 3 8B to generate Nepali, with some forgetting [10]; and in this journal, Tibetan-LLaMA 2 adapted an open model to Tibetan [45]. All of these concern native-script text.

2.5 Evaluating Adapted Models

Generated text is usually compared with references using BLEU [32], chrF and chrF++ [35, 36], ROUGE [26] or embedding similarity such as BERTScore [49]. Character-level chrF credits partial matches within words, which suits morphologically rich languages [35], but all of these metrics reward closeness to one reference. Likelihood-based measures instead evaluate a model on the reference itself. Per-token perplexity depends on the tokenizer, whereas bits per byte normalizes the total negative log-likelihood by the length of the text in bytes, which makes it comparable across tokenizers [11]. The two kinds of measure can diverge: a model that assigns high probability to human text may still generate text with different statistical properties [28]. Comparisons also need explicit uncertainty. Fine-tuning results vary substantially with the random seed [8]; differences between systems evaluated on one test set should be assessed with resampling or significance tests [9, 24]; and metric configurations should be reported so that scores can be reproduced [37].

2.6 The Closest Benchmark

Rimal and Rimal [41] fine-tuned Llama-3.1-8B, Mistral-7B-v0.1 and Qwen3-8B on 10,000 Romanized Nepali instruction examples built from the source dataset used here, with half of the instructions machine-translated into English and half transliterated. They trained one QLoRA model with rsLoRA per architecture and reported perplexity, BERTScore, chrF++, ROUGE and BLEU, together with a qualitative case study. All three models adapted, and Qwen3-8B obtained the highest fine-tuned chrF++ (27.47, against 26.97 for Llama-3.1-8B and 23.95 for Mistral-7B-v0.1). They defined adaptation headroom as the fine-tuned minus the zero-shot value of a metric, argued that models with weaker zero-shot

performance have more "representational plasticity", and attributed Llama's difficulty to "its Tiktoken tokenizer over-fragmenting Romanized Nepali into low-frequency subword units", leaving the measurement of fertility to future work. Their study leaves open how smaller models compare, how much results vary across training runs, whether directly measured fertility supports the over-fragmentation account, and how likelihood compares across models with different tokenizers. We are not aware of a study that measures tokenizer fertility, byte-normalized reference likelihood and generation overlap together for small models adapted to Romanized Nepali, with seed-level uncertainty. Our recipe follows theirs (Section 4.2), but model scale, data processing, checkpoint selection, decoding and the likelihood measure differ, so we compare findings, not scores.

3 Data

3.1 Source, Filtering and Splits

The source is the Nepali instruction dataset Saugatkafley/alpaca-nepali-sft on the Hugging Face Hub, with 52,005 examples whose instruction, input and output fields are in Devanagari. Its dataset card states no licence and gives little information on provenance, but the data are a Nepali translation of the Stanford Alpaca data [47], which is released under CC BY-NC 4.0. The type A instructions of our test split (Section 3.3), translated back into English, match Alpaca instructions closely: 29.8% are identical to one of its 52,002 instructions, and 92.2% have a normalized edit similarity of at least 0.8 to the nearest one. We follow the data recipe of Rimal and Rimal [41] with our own transliteration choices. We removed examples that still contained Devanagari characters after transliteration and examples whose outputs were shorter than three words, which left 48,106 eligible examples; examples with an empty input field were kept. From these we drew 10,500 examples (sampling seed 42) and split them into 9,000 training, 500 validation and 1,000 test examples, with the two instruction types (Section 3.3) balanced 50/50 in every split. The 500-example validation split, which the prior recipe does not describe, accounts for the difference from its 10,000 examples.

Empty inputs occur in 5,576 training, 314 validation and 660 test examples. Split files are identified by SHA-256 prefixes (train 9ed86da7…, validation 3f7f6854…, test d656353e…; full hashes in the code repository). We found no exact instruction overlap between splits. One output is shared between the training and validation splits and three between the training and test splits, each of three or more words, and we kept them. We also kept two artifacts of the source data: placeholder strings meaning "no input" in 114 training, 5 validation and 14 test examples, and zero-width-space characters in 113, 11 and 12 examples.

3.2 Romanization

Romanized text was produced by ITRANS transliteration of the Devanagari fields, followed by lowercasing. Rimal and Rimal [41] describe their transliteration as using an Indic transliteration library with phonetic variation permitted, so the scheme is our choice. Table 1 shows how it treats some common words. It has three properties that matter for interpreting the results.

First, it is deterministic: every Devanagari word has exactly one romanized spelling, so the benchmark contains none of the spelling variation of naturally typed Romanized Nepali.

Second, lowercasing removes the case distinctions that ITRANS uses to encode Devanagari letters, which merges 11 groups of letters: eight consonant groups (च/छ, त/ट, थ/ठ, द/ड, ध/ढ, न/ण, श/ष and ङ/ञ) and three vowel groups (अ/आ, इ/ई and उ/ऊ, with their vowel signs; Appendix B). Some merged pairs are contrastive in spoken Nepali, such as unaspirated and aspirated affricates (च/छ) and dental and retroflex stops (त/ट, द/ड), whereas the long and short vowels are distinguished only in the script [22]. To estimate how much text the merger affects, we transliterated a public sample of 1,233 Nepali sentences (Section 5.1 and Appendix B). In this sample, 58.8% of running words contain at least one uppercase ITRANS character, so lowercasing changes the spelling of most words, and 3.8% of the resulting word types merge two or more distinct ITRANS spellings. These merged types are frequent: they cover 13.8% of running words, as with पनि and पानी (both *pani*) or तिन, तीन and टिन (all *tina*).

Third, the scheme keeps the inherent vowel at the end of a word, which conventional romanizations often drop: नेपाल is conventionally written *Nepal*, but the scheme gives *nepala*. In the public sample, 36.2% of running words end in an inherent vowel, and 29.4% are words of two or more aksharas (orthographic syllables) that do so. For the fertility analysis we therefore also use two alternative conventions: case-preserving ITRANS (R1) and the benchmark scheme without the final inherent vowel in words of two or more aksharas (R3), a simple approximation of informal spelling. The benchmark scheme itself is R2.

Table 1: Example words under the three romanization conventions. R2 (lowercased ITRANS) is the benchmark scheme; R1 and R3 are used only in the fertility analysis (Section 5.1). Under R2, पनि and पानी, तीन and टिन, and मन and मान become identical.

Devanagari	Gloss	R1: ITRANS	R2: lowercased	R3: R2, final vowel dropped
नेपाल	Nepal	`nepAla`	`nepala`	`nepal`
घर	house	`ghara`	`ghara`	`ghar`
छ	is	`Cha`	`cha`	`cha`
पनि	also	`pani`	`pani`	`pani`
पानी	water	`pAnI`	`pani`	`pani`
तीन	three	`tIna`	`tina`	`tin`
टिन	tin, can	`Tina`	`tina`	`tin`
मन	mind	`mana`	`mana`	`man`
मान	respect	`mAna`	`mana`	`man`
काठमाडौं	Kathmandu	`kAThamADauM`	`kathamadaum`	`kathamadaum`
मातृभूमि	motherland	`mAtRRibhUmi`	`matrribhumi`	`matrribhumi`

3.3 Instruction Formats

Following the prior recipe [41], each example has one of two formats. In type A, the instruction was machine-translated from Nepali into English and the input and output were transliterated; in type B, all three fields were transliterated. Type A instructions have therefore been translated twice, from English into Nepali in the source dataset and back into English here, and back-translation sometimes changes the meaning: one test instruction asks why chloroplasts are found "in plant cells and animal cells", where the Alpaca original has "in plant cells and not animal cells". Prompts follow the Alpaca template [47], with its no-input variant for examples without an input, and the target is always the Romanized Nepali output.

3.4 Length

Training sequences (prompt plus response) were limited to 512 tokens, which truncated 178 (Qwen2.5-3B), 164 (Llama-3.2-3B) and 127 (Gemma-2-2B) of the 9,000 training examples (1.4–2.0%) and 6, 6 and 5 of the 500 validation examples. In the test set, 19, 17 and 14 examples exceed 512 tokens under the same tokenizers, but no generation prompt does (Section 4.5). The 1,000 test references contain 41,388 whitespace-delimited words, 41.4 per response on average, and 124 (Qwen2.5-3B), 117 (Llama-3.2-3B) and 88 (Gemma-2-2B) of them are longer than the 256-token generation limit.

4 Experimental Setup

4.1 Models

All models are base checkpoints that were not instruction-tuned: meta-llama/Llama-3.2-3B [30], Qwen/Qwen2.5-3B [38] and google/gemma-2-2b [12] in the main comparison, and meta-llama/Llama-3.1-8B [14, 29] in the supplementary within-family comparison (Table 2; revisions in Appendix A). The three primary models differ in more than the tokenizer. Gemma-2-2B has about 2.0 billion non-embedding parameters against 2.8 billion for the other two, saw fewer pretraining tokens, and puts 22.6% of its parameters into the embeddings of its 256k-entry vocabulary. Llama-3.2-3B and Gemma-2-2B were trained with knowledge distillation from larger models [12, 30].

4.2 Adaptation Recipe

Each model is adapted with QLoRA [6] through Unsloth [15]. Base weights are loaded in 4-bit NF4 with double quantization and fp16 compute, and rank-stabilized LoRA adapters [20] ($r = 32$, $\alpha = 64$, dropout 0, no bias) are trained on the attention (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and feed-forward (`gate_proj`, `up_proj`, `down_proj`) projections. With rank-stabilized scaling the adapter output is multiplied by $\alpha/\sqrt{r} = 64/\sqrt{32} \approx 11.3$, which is 5.7 times the standard LoRA factor $\alpha/r = 2$ [16]. The adapters have 59.9M trainable parameters for Qwen2.5-3B, 48.6M for Llama-3.2-3B, 41.5M

Table 2: Models. Parameter counts are computed from the published model configurations; the embedding share of Llama-3.1-8B includes its separate output matrix. Vocabulary, pretraining tokens and licences are from the model reports and cards [12, 14, 29, 30, 38]. The LoRA adapter sizes implied by these configurations match the trainable-parameter counts in Section 4.2.

Model	Total params	Non-embedding params	Embedding share	Vocabulary	Tokenizer	Pretraining tokens	Licence
Llama-3.2-3B	3.21B	2.82B	12.3%	128,256	byte-level BPE	up to 9T	Llama 3.2 Community License
Qwen2.5-3B	3.09B	2.77B	10.1%	151,643	byte-level BPE	18T	Qwen Research License
Gemma-2-2B	2.61B	2.02B	22.6%	256k	SentencePiece	2T	Gemma Terms of Use
Llama-3.1-8B	8.03B	6.98B	13.1%	128,256	byte-level BPE	~15T	Llama 3.1 Community License

for Gemma-2-2B and 83.9M for Llama-3.1-8B (1.0–1.9% of the total). Unsloth substitutes its own pre-quantized 4-bit checkpoints where they exist (the Llama-3.2-3B zero-shot log shows `unsloth/Llama-3.2-3B-unsloth-bnb-4bit`) and leaves some modules unquantized, depending on the model: `lm_head` and the MLPs of layers 1, 2, 3, 28 and 30 for Qwen2.5-3B, `lm_head` and the layer-1 MLP for Llama-3.2-3B, and none for Gemma-2-2B (measured with Unsloth 2026.11.2; runs under other versions may differ).

Training uses a learning rate of 1×10^{-4} with cosine decay and 200 warm-up steps, 8-bit AdamW, three epochs (3,375 optimizer steps), a maximum sequence length of 512 tokens without packing, fp16, and an effective batch size of 8 (2 per device $\times$ 4 accumulation steps) on a single NVIDIA T4 GPU. Each model is trained with seeds 42, 43 and 44. The recipe follows Rimal and Rimal [41] except for the batch size and the validation split. It was shared across models and tuned for none of them, so it may suit some models better than others. For Gemma-2-2B, Unsloth compilation was disabled (`UNSLOTH_COMPILE_DISABLE=1`) in training and evaluation for all three seeds.

4.3 Checkpoint Selection and Divergence

We evaluate on the validation split every 100 steps and keep the checkpoint with the lowest validation loss; training is not stopped early. Appendix A lists every training run and its status. A second Qwen2.5-3B seed-42 run, launched because of a Kaggle session limit and to keep package versions the same across seeds, diverged: its validation loss jumped from 1.46 at step 1200 to 6.99 at step 1300, 4.8 times the best earlier value, before recovering. After it, we defined a run as diverged if any validation loss after step 500 exceeded 1.5 times the best earlier validation loss, and we report the first non-diverged run for each seed. The rule uses validation loss only, but it was fixed after the diverged run's test scores had been seen, so it is a post hoc diagnostic rather than a pre-specified exclusion rule. No other run met it. The maximum post-step-500 ratio was 1.14–1.15 for Gemma-2-2B and 1.107–1.115 for Llama-3.1-8B; the retained records do not give the maxima for the other runs, and they give the selected step only for Qwen2.5-3B seeds 43 and 44 (step 2200 in both). Llama-3.2-3B seed 43 logged one non-finite gradient norm at step 2150, and training continued normally.

4.4 Zero-Shot Conditions

The zero-shot conditions use the base checkpoints without adapters under two prompts: (i) the Alpaca template used in fine-tuning, with its no-input variant where there is no input, and (ii) a bare prompt consisting of the instruction followed by the input on a new line. Each condition is a single deterministic run. The base models were not instructed to answer in Romanized Nepali, so these conditions measure what the models do with the task as given, not their best attainable zero-shot performance.

4.5 Decoding and Likelihood

All conditions share decoding and scoring. We decode greedily with at most 256 new tokens and a repetition penalty of 1.1, in fp16 with batch size 8 and left padding; for fine-tuned models the adapter is applied to the 4-bit base without merging. Prompts are truncated to 512 tokens, a limit that no test prompt reached.

We also score each reference response given its prompt by teacher forcing. For test item i, let NLL_i be the summed negative log-likelihood, in nats, of the reference response tokens, excluding the end-of-sequence token, and let B_i be the length of the reference in UTF-8 bytes. Prompt and response together are capped at 1,000 tokens, which truncated one item (id 289) under the Qwen and Llama tokenizers. Over the $N = 1{,}000$ test items, bits per byte is

$$\mathrm{BPB} = \frac{\sum_{i=1}^{N} \mathrm{NLL}_i}{\ln 2 \sum_{i=1}^{N} B_i} \tag{1}$$

Because the denominator counts bytes rather than tokens, BPB can be compared across models with different tokenizers [11]; lower values mean that the model assigns higher probability to the references. We also compute reference token perplexity, the exponential of the pooled NLL per response token, but compare it only within a model. BPB under the two zero-shot prompts is not compared, because the prompts condition the model differently.

4.6 Generation Metrics

We report corpus-level BLEU [32] and chrF++ (character n-grams with word order 2) [36], computed with sacreBLEU 2.6.0 [37] on a 0–100 scale (signatures in Appendix A); ROUGE-1, ROUGE-2 and ROUGE-L F-scores [26], computed with the rouge-score package without stemming; and BERTScore F1 [49] with multilingual BERT (bert-base-multilingual-cased [7]; bert-score 0.3.13, no baseline rescaling), averaged over items. Each test item has one reference. The BERTScore encoder was pretrained on text in which Nepali appears in Devanagari and has not been validated for Romanized Nepali, so we treat BERTScore as a supplementary similarity score. Empty generations are kept in every metric and score 0 in BERTScore; for runs with empty generations we also give the BERTScore mean over non-empty outputs as a descriptive supplement. Recomputing chrF++, BLEU, ROUGE and BPB from the saved per-item outputs reproduces the values of the evaluation scripts for every run (Appendix C); BERTScore could not be recomputed in our analysis environment.

In our environment (bert-score 0.3.13 with Transformers 5.17.0), the evaluation script failed at the BERTScore step for zero-shot runs that contained at least one empty generation, after generation and likelihood scoring had finished. For these runs we rebuilt all text metrics from the saved outputs with a separate script that reuses the saved NLL values and bootstrap settings and does not regenerate. On the Qwen2.5-3B Alpaca run, which had no empty outputs, this script reproduced BLEU, chrF++, ROUGE and BPB exactly and BERTScore to within 10^{-7}.

4.7 Uncertainty

We report two kinds of uncertainty. Variation across training seeds is given as the mean and sample standard deviation (SD) over three seeds; with three runs the SD itself is imprecise, since a 95% interval for the true SD runs from 0.52 to 6.3 times the sample SD. Test-set sampling variation is estimated by resampling the 1,000 test items from the saved outputs, which hold each generation, its reference and the reference NLL. Per-run 95% percentile intervals use 2,000 resamples (seed 0). To compare two models, we resample the items once per replicate (2,000 replicates, seed 0), recompute corpus chrF++, BLEU and pooled BPB for every seed of both models on the resampled items, average over each model's seeds and take the difference. These paired item-level intervals condition on the trained seeds. As a complement that conditions on the test set instead, Table 9 also gives seed-level intervals (Welch's *t* over the three per-seed scores of each model). Fertility intervals use 1,000 bootstrap resamples over references (seed 0) and paired fertility differences 2,000 resamples (seed 1). The evaluation scripts' own per-run intervals used 200 resamples (Python random seed 0); the intervals reported here replace them. We do not interpret differences that neither kind of interval resolves.

4.8 Output Audit

One script (SHA-256 69b88899…) audits every saved output. It flags (i) English-like output, when English stopwords make up at least 15% of alphabetic tokens, using a fixed 111-word list screened against the reference responses; (ii) input copying, when the first non-empty line, of at least three words, reproduces a non-template line of the prompt; and (iii) repetition loops, when at least half of the non-empty lines repeat earlier lines, for outputs of at least three lines. The English and loop checks apply only to outputs long enough to evaluate, so their denominators vary. The same definitions apply to all 20 evaluation runs. As a descriptive baseline, the audit of the 1,000 reference responses flags none of 922 evaluable responses as English-like, 10 as input copies and 1 of 181 evaluable as a repetition loop. Because the stopword list was screened against these references, the baseline does not validate the English flag independently. The flags are surface heuristics, not judgments of correctness: English words can be appropriate in code or explanations, and repeated lines can occur in valid structured answers.

4.9 Software and Provenance

The primary fine-tuned evaluations used evaluation script v2.2 and the Llama-3.1-8B evaluations v2.4 (51e1bd41…). The fine-tuned code path (prompt template, generation settings, likelihood scoring and metrics) is identical in the two

Table 3: Tokenizer fertility on the 1,000 benchmark references (41,388 words) beside the three-seed fine-tuned results. Fertility is the mean over references of tokens per whitespace-delimited word, with 95% bootstrap intervals (1,000 resamples). Higher chrF++ and lower BPB are better. Mistral-7B-v0.1 was not trained here.

Tokenizer	Tokens per word [95% CI]	Median	Tokens per char	Fine-tuned chrF++	Fine-tuned BPB
Gemma-2-2B	2.717 [2.689, 2.747]	2.677	0.341	29.02	0.842
Llama-3.2-3B (= Llama-3.1-8B)	2.938 [2.908, 2.968]	2.928	0.374	30.51	0.779
Qwen2.5-3B (= Qwen3-8B)	2.997 [2.965, 3.029]	3.000	0.380	30.61	0.821
Mistral-7B-v0.1	3.295 [3.262, 3.329]	3.276	0.419	–	–

Table 4: Fertility on a public sample of 1,233 Nepali sentences under three romanization conventions: R1, case-preserving ITRANS; R2, lowercased ITRANS (the benchmark scheme); R3, R2 without the final inherent vowel in words of two or more aksharas. Word shares are over 8,840 running words under R2. Bootstrap intervals have a half-width of 0.03–0.04 under every convention.

Tokenizer	R1	R2 [95% CI]	R3	R2 single-token words	R2 words of $\geq$3 tokens	R2 tokens per char
Gemma-2	2.493	2.135 [2.107, 2.165]	2.069	30.9%	31.6%	0.339
Llama 3	2.712	2.358 [2.332, 2.385]	2.264	22.4%	39.7%	0.377
Qwen2.5	2.733	2.387 [2.359, 2.414]	2.294	22.2%	40.9%	0.381
Mistral v0.1	3.053	2.648 [2.612, 2.683]	2.536	19.0%	49.7%	0.420

versions; v2.4 adds the two zero-shot prompt modes, saves generations before computing metrics, and corrects a field that left the v2.2 metric files recording their script hash as unknown. The zero-shot runs of the primary models used one script (33e10eac…) with per-run overrides; Appendix A lists which patched copies have recorded hashes. Package versions differed across sessions (Appendix A). To check that this does not change results, we re-evaluated Qwen2.5-3B seed 43 in a later environment (Unsloth 2026.11.2, Transformers 5.17.0, PEFT 0.21.2 and PyTorch built for CUDA 13.0): all 1,000 generations, all per-item NLL values and every metric were identical. This check covers one adapter. The training and evaluation code, the re-analysis of the saved outputs (Sections 5.2–5.6 and Appendix C), the tokenizer and romanization analysis (Section 5.1 and Appendix B) and the figure code were written with the assistance of an AI model (Acknowledgments).

5 Results

5.1 Tokenizer Fertility on Romanized Nepali

On the 1,000 test references (41,388 words), each encoded whole without special tokens, the Gemma-2-2B tokenizer has the lowest fertility, 2.717 tokens per word, followed by Llama-3.2-3B (2.938) and Qwen2.5-3B (2.997) (Table 3). Medians and tokens per character give the same order, and paired bootstrap differences confirm it: Llama exceeds Gemma by 0.221 [0.206, 0.235], Qwen exceeds Llama by 0.059 [0.051, 0.067], and Qwen exceeds Gemma by 0.280 [0.268, 0.292]. Per reference, Llama's tokenizer needed fewer tokens per word than Gemma's for only 9.1% of the references, and Qwen's for only 4.5%. The Mistral-7B-v0.1 tokenizer, which we did not fine-tune, splits words more (3.295). The Llama-3.1-8B and Llama-3.2-3B tokenizers produced identical token IDs for all 1,000 references, as did the Qwen2.5-3B and Qwen3-8B tokenizers. Among the trained models the range is narrow: 2.72–3.00 tokens per word, a 10% spread.

Because the benchmark references are themselves in one romanization scheme, we also measured fertility on public Nepali text under the three conventions of Section 3.2 (Table 4; details in Appendix B). The sample comprises 1,233 sentences: 1,146 from the Mozilla Common Voice Nepali text corpus [2] and 87 lines of the Universal Declaration of Human Rights in Nepali distributed with NLTK [4]. Under the benchmark scheme (R2), tokens per character on this sample (0.339–0.420) are close to those on the benchmark references (0.341–0.419), but fertility is lower (2.13–2.65), because the sample's words are shorter (about 6.4 against 7.9 characters per word, counting the following space). The tokenizer files used for this sample reproduce the Hugging Face token counts exactly on all 1,000 benchmark references (Appendix B).

The order Gemma < Llama < Qwen < Mistral held in all 2,000 paired bootstrap resamples under every convention. The convention, however, changed fertility about as much as the choice of tokenizer did. Lowercasing (R1 to R2) lowered fertility by 12.7–14.4%, or 0.35–0.41 tokens per word, which approaches the full spread across the four tokenizers

Table 5: Zero-shot results on the 1,000 test items (one deterministic run per condition). Empty generations are kept and score 0 in BERTScore; the mean over non-empty outputs is in parentheses. BPB is comparable across models within a prompt, not across prompts.

Model	Prompt	chrF++	BLEU	BERTScore F1	Empty outputs	BPB
Qwen2.5-3B	Alpaca	13.65	0.64	0.545	0	2.373
Qwen2.5-3B	bare	11.45	0.43	0.492 (0.541)	91	2.404
Llama-3.2-3B	Alpaca	12.51	1.01	0.587	4	2.225
Llama-3.2-3B	bare	12.65	0.38	0.536	9	2.276
Gemma-2-2B	Alpaca	10.13	0.41	0.539	3	2.415
Gemma-2-2B	bare	10.96	0.12	0.526	2	2.466

under R2 (0.51) and is more than ten times the Llama–Qwen difference (0.029). Dropping the final inherent vowel (R2 to R3) lowered it by a further 3.1–4.2%. Gemma's lower fertility comes from keeping more words whole: under R2, 30.9% of running words are a single Gemma token, against 19.0–22.4% for the other tokenizers, and 31.6% are split into three or more tokens, against 39.7–49.7%.

Fertility did not correspond to the fine-tuned results (Table 3). Gemma-2-2B, whose tokenizer splits words least, has the lowest overlap scores and the highest BPB of the three models (Section 5.3), and Llama-3.2-3B has the lowest BPB although its tokenizer splits words more than Gemma's. With three trained models that also differ in size, data and architecture (Table 2), this is a counterexample to choosing models by fertility, not an estimate of its effect.

5.2 Zero-Shot Behaviour

Without adaptation, all three base models score far below their fine-tuned versions (Table 5). chrF++ is 10.1–13.7, against 29.0–30.6 after fine-tuning, and BPB under the Alpaca prompt is 2.23–2.41, against 0.78–0.84. Most zero-shot outputs are not Romanized Nepali at all: 543–923 of 1,000 outputs per condition (54.3–92.3%) are flagged as English-like, against none of the evaluable references (Table 6).

The effect of the prompt depends on the model. For Qwen2.5-3B the Alpaca prompt gives higher chrF++ (13.65 [13.32, 13.98] against 11.45 [11.04, 11.86]) and BERTScore, and a higher BLEU point estimate with overlapping intervals (0.64 [0.52, 0.77] against 0.43 [0.29, 0.60]). For Llama-3.2-3B, chrF++ is similar under the two prompts (12.51 and 12.65), while BLEU and BERTScore are higher with the Alpaca template. Gemma-2-2B has lower chrF++ under the Alpaca prompt (10.13 against 10.96) but higher BLEU and BERTScore. We report these as point estimates and do not claim that the smaller differences are reliable. Qwen2.5-3B's bare-prompt BERTScore of 0.492 includes 91 empty generations scored as 0; over its 909 non-empty outputs the mean is 0.541, close to its Alpaca value. Likelihood already orders the base models: under both prompts, BPB is lowest for Llama-3.2-3B and highest for Gemma-2-2B, and every paired difference is resolved.

The failures differ by model and prompt (Table 6). Llama-3.2-3B copied the prompt in 135 outputs under the Alpaca template, mostly on type B items (24.2% of type B outputs), and produced 110 repetition loops under the bare prompt (16.4% of type B outputs). Qwen2.5-3B produced 91 empty generations and 29 loops under the bare prompt, against none and 2 under the Alpaca template. Gemma-2-2B produced few copying or repetition flags under either prompt, and the most English-like output across the two prompts (830 and 877). Where the audit breaks failures down by type, they concentrate on type B prompts, whose instructions are themselves Romanized. After fine-tuning, the nine runs had 0–2 English-like flags, 4–20 input copies (0.4–2.0%), 0–2 loops and no empty generations, rates comparable to those of the references. The flagged fine-tuned outputs that we inspected mostly answered prompts whose references are themselves structured or code (an HTML table, SQL, JavaScript or numbered lists) or contain quoted English words.

5.3 Adaptation across Model Families

All three models adapt strongly (Table 7). Mean chrF++ rises to 29.02–30.61 and BPB falls to 0.779–0.842, with no empty outputs and few audit flags. Qwen2.5-3B and Llama-3.2-3B cannot be separated on any generation metric: their mean chrF++ differs by 0.09 points, BLEU by 0.04 and ROUGE-1 by 0.001, and their seed ranges overlap (chrF++ 29.53–31.42 and 29.97–31.38). Gemma-2-2B is lower on every overlap metric. Its BLEU separates cleanly, with all three seeds (4.51–4.99) below every Qwen and Llama seed (5.61 or above), whereas its chrF++ overlaps: its best seed (29.73) exceeds the lowest Qwen seed (29.53). BERTScore differs little between models (0.750–0.757). All three models score higher on type A than on type B, by 2.4–2.8 chrF++ points (type A 30.25–31.92, type B 27.83–29.40), and BPB is lower on type A in every model.

Table 6: Output audit (heuristic flags, not quality judgments). Counts are out of 1,000 outputs unless an evaluable denominator is shown; the English and loop checks apply only to outputs long enough to evaluate.

Condition	English-like	Input copy	Repetition loop	Empty
Reference responses	0 / 922	10	1 / 181	0
Fine-tuned (9 runs)	0–2 / 909–931	4–20	0–2 / 175–210	0
Qwen2.5-3B, Alpaca	923	2	2	0
Qwen2.5-3B, bare	608	3	29	91
Llama-3.2-3B, Alpaca	543	135	24	4
Llama-3.2-3B, bare	641	22	110 / 794	9
Gemma-2-2B, Alpaca	830	3	0	3
Gemma-2-2B, bare	877	1	6	2

Table 7: Fine-tuned results on the 1,000 test items: mean $\pm$ sample SD over seeds 42, 43 and 44. Higher is better except for BPB. Qwen2.5-3B seed 42 is the first non-diverged run (Section 4.3).

Model	chrF++	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1	BPB
Qwen2.5-3B	30.61 ± 0.97	5.86 ± 0.26	0.252 ± 0.004	0.082 ± 0.001	0.203 ± 0.002	0.757 ± 0.001	0.821 ± 0.003
Llama-3.2-3B	30.51 ± 0.76	5.81 ± 0.10	0.251 ± 0.005	0.080 ± 0.003	0.203 ± 0.004	0.755 ± 0.001	0.779 ± 0.001
Gemma-2-2B	29.02 ± 0.62	4.78 ± 0.24	0.220 ± 0.004	0.064 ± 0.005	0.176 ± 0.004	0.750 ± 0.001	0.842 ± 0.002

Gains over the zero-shot baselines depend on the baseline prompt (Table 8, Figure 1). Measured from the Alpaca-prompt baseline, chrF++ rises by 16.95 (Qwen2.5-3B) to 18.89 (Gemma-2-2B) points among the small models and by 21.30 for Llama-3.1-8B. Measured from the bare-prompt baseline, Qwen2.5-3B gains most (19.16) and Llama-3.1-8B least (17.77). The models with the largest and smallest gains thus swap places. Because each model's fine-tuned score is the same in both columns, these changes come entirely from the zero-shot runs. Under the bare prompt, three of the four gains lie within 0.3 points of one another, well inside seed variation, and BLEU gains give yet another order under each prompt. Likelihood gives a steadier picture: every model's fine-tuned BPB is 0.341–0.358 of its zero-shot value under either prompt, a 64–66% reduction, and the BPB order of the four models is the same before and after fine-tuning.

5.4 Training Stability and Seed Variation

Seed variation is of the same size as test-set variation. Seed SDs of chrF++ are 0.62–0.97 points, and the 95% bootstrap interval of a single fine-tuned run is 1.7–2.0 points wide. Table 9 shows what the two kinds of interval can resolve. Paired item-level comparisons resolve Gemma-2-2B's deficits in chrF++ and BLEU against both other small models and Llama-3.1-8B's small gains over Llama-3.2-3B, but not the difference between Llama-3.2-3B and Qwen2.5-3B (chrF++ -0.09 [-0.50, 0.31]). Seed-level intervals are wider: they resolve no chrF++ difference, and for BLEU only Gemma-2-2B's deficits and the 8B gain. Differences of about one chrF++ point between single runs are therefore not interpretable, and even three-seed means need a paired test. Training was stable apart from the one diverged Qwen2.5-3B replicate (Section 4.3). After step 500, validation loss rose to at most 1.15 times its best earlier value for Gemma-2-2B and 1.115 times for Llama-3.1-8B; selection on the validation split keeps the earlier, better checkpoint.

5.5 Reference Likelihood against Generation Overlap

Likelihood and generation overlap give different pictures of how the adapted models compare (Figure 2, Table 9). Every BPB difference between models is resolved by both kinds of interval, and the order holds in every seed. At the item

Table 8: Gains from zero-shot to fine-tuned: the three-seed fine-tuned mean minus the single zero-shot run under each prompt, and the ratio of fine-tuned to zero-shot BPB under each prompt. Computed from Tables 5, 7 and 10 before rounding.

Model	chrF++ gain, Alpaca	chrF++ gain, bare	BLEU gain, Alpaca	BLEU gain, bare	BPB ratio, Alpaca	BPB ratio, bare
Llama-3.1-8B	+21.30	+17.77	+4.99	+6.00	0.358	0.349
Gemma-2-2B	+18.89	+18.05	+4.37	+4.66	0.349	0.341
Llama-3.2-3B	+18.00	+17.86	+4.80	+5.43	0.350	0.342
Qwen2.5-3B	+16.95	+19.16	+5.22	+5.42	0.346	0.342

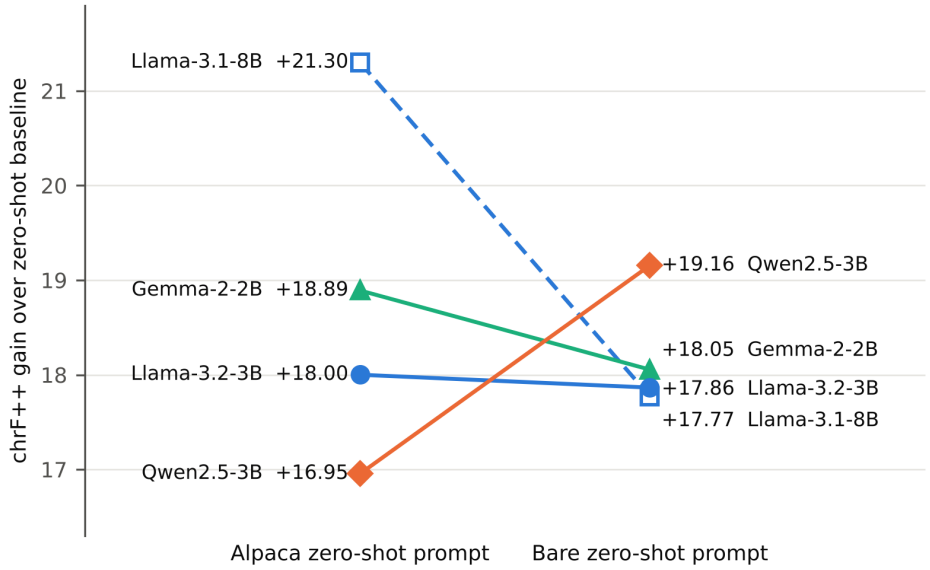


Figure 1: Gain in chrF++ from zero-shot to fine-tuned, measured from the Alpaca-prompt and the bare-prompt zero-shot baselines. Lines connect the same model. Each model's fine-tuned score is the same in both columns; only the baseline changes.

Table 9: Differences between models (first model minus second) with two kinds of 95% interval. Paired item-level: the 1,000 test items resampled 2,000 times, scores recomputed for every seed and averaged over each model's seeds; it conditions on the trained seeds. Seed-level: Welch intervals over the three per-seed scores of each model; it conditions on the test set. Negative BPB differences mean higher likelihood for the first model. Intervals in bold exclude zero.

Comparison	Measure	Difference	Paired item-level 95% CI	Seed-level 95% CI
Llama-3.2-3B − Qwen2.5-3B	BPB	−0.0423	**[−0.0462, −0.0387]**	**[−0.0493, −0.0353]**
	chrF++	−0.09	[−0.50, 0.31]	[−2.12, 1.93]
	BLEU	−0.04	[−0.56, 0.39]	[−0.62, 0.53]
Gemma-2-2B − Qwen2.5-3B	BPB	+0.0205	**[0.0168, 0.0244]**	**[0.0142, 0.0268]**
	chrF++	−1.59	**[−2.01, −1.14]**	[−3.58, 0.40]
	BLEU	−1.08	**[−1.51, −0.68]**	**[−1.65, −0.50]**
Gemma-2-2B − Llama-3.2-3B	BPB	+0.0628	**[0.0580, 0.0684]**	**[0.0584, 0.0672]**
	chrF++	−1.50	**[−1.92, −1.06]**	[−3.09, 0.10]
	BLEU	−1.03	**[−1.34, −0.75]**	**[−1.56, −0.51]**
Llama-3.1-8B − Llama-3.2-3B	BPB	−0.0530	**[−0.0559, −0.0501]**	**[−0.0571, −0.0490]**
	chrF++	+0.68	**[0.26, 1.10]**	[−0.98, 2.33]
	BLEU	+0.58	**[0.17, 1.09]**	**[0.21, 0.95]**

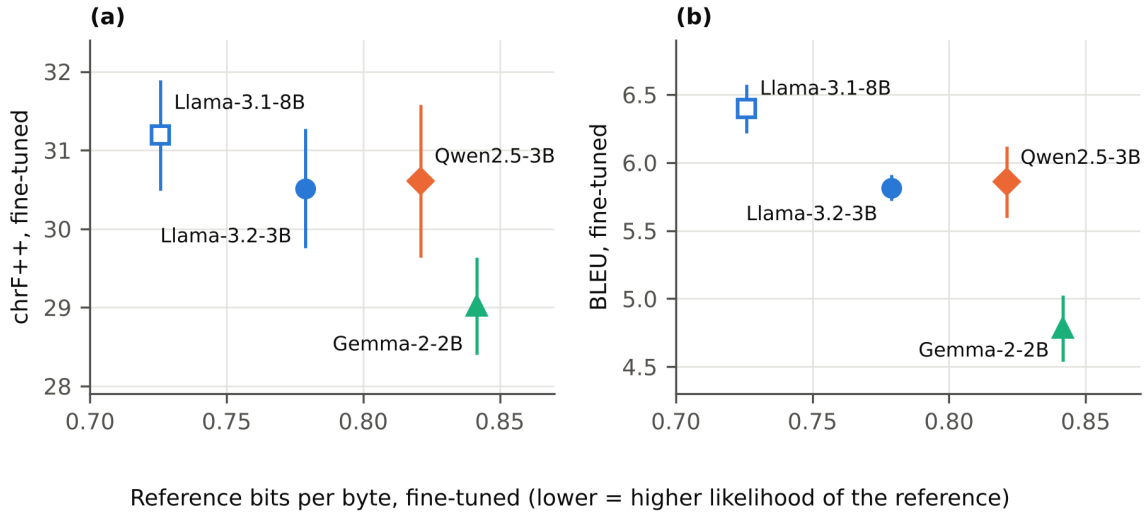


Figure 2: Fine-tuned reference likelihood (BPB; lower means higher likelihood) against (a) chrF++ and (b) BLEU. Points are three-seed means and bars show ±1 seed SD. Llama-3.2-3B and Qwen2.5-3B differ clearly in BPB but in neither generation metric.

level, the generation metrics separate every pair except one: Llama-3.2-3B and Qwen2.5-3B, the pair with the largest BPB gap among the small models (0.0423), cannot be separated on chrF++ or BLEU, nor on ROUGE or BERTScore. Gemma-2-2B's BPB deficit relative to Qwen2.5-3B is half as large (0.0205) but comes with the largest generation gap (−1.59 chrF++, −1.08 BLEU). The size of a likelihood gap therefore predicted neither the size nor the presence of a generation gap. By likelihood, Llama-3.2-3B is the best of the three small models; by generation overlap, it ties with Qwen2.5-3B. The two kinds of measure agree only on placing Gemma-2-2B last. Output length does not explain the pattern. Fine-tuned outputs were shorter than the references for every model (78–79% of their length in characters on average), 3–8% of them ran to the 256-token limit, and the two models that tie on generation also match on length (both 79%). Because BPB is normalized by bytes, these differences are not an artefact of how many tokens each tokenizer produces, although they may still reflect tokenization together with the models' other differences.

5.6 Within-Family Comparison with Llama-3.1-8B

Llama-3.1-8B tokenizes the references identically to Llama-3.2-3B, which removes segmentation as a source of difference between the two, although they differ in model generation, training data and distillation as well as size (Table 2). After fine-tuning (Table 10), the 8B model has lower BPB in every seed, by 0.0530 [0.0501, 0.0559] in the paired comparison. Its gains in generation are small: 0.68 chrF++ and 0.58 BLEU points. Both are resolved by the paired item-level test (chrF++ [0.26, 1.10], BLEU [0.17, 1.09]), but only the BLEU gain is consistent across seeds: every 8B seed has higher BLEU than every 3B seed (6.20–6.54 against 5.72–5.91), whereas the 3B seed-43 run (chrF++ 31.38) scores above two of the three 8B runs. BERTScore differs by 0.005. The type A advantage holds in every 8B seed (chrF++ 32.47, 32.00 and 33.30 on type A against 29.15, 29.60 and 30.74 on type B). The audit shows no empty generations and no Devanagari output in the three 8B runs, with 0, 3 and 0 English-like outputs, 9, 13 and 9 input copies and 2, 2 and 4 repetition loops, similar to Llama-3.2-3B.

Without fine-tuning the likelihood gap is larger, but generation does not follow it. The 8B model's BPB is lower by 0.198 [0.192, 0.203] under the Alpaca prompt and by 0.194 [0.189, 0.199] under the bare prompt, while its chrF++ is lower under the Alpaca prompt (9.89 against 12.51) and higher under the bare prompt (13.42 against 12.65). Under the Alpaca prompt the 8B model returned 141 empty generations, 128 of them on type B items, against 4 for the 3B model; we did not investigate the cause. In sum, the larger model assigns higher probability to the references before and after fine-tuning, but after fine-tuning its outputs are only slightly closer to them, by 0.68 chrF++ and 0.58 BLEU points, and only the BLEU gain is consistent across seeds.

Table 10: Llama-3.1-8B against Llama-3.2-3B. Fine-tuned rows: mean ± sample SD over seeds 42, 43 and 44 (8B per seed: chrF++ 30.79, 30.78 and 32.00; BLEU 6.20, 6.46 and 6.54; BPB 0.7263, 0.7274 and 0.7236). Zero-shot rows: single runs, with the BERTScore mean over non-empty outputs in parentheses. BPB is comparable across models within a condition, not across zero-shot prompts.

Condition	Model	chrF++	BLEU	BERTScore F1	BPB	Empty
Fine-tuned	Llama-3.1-8B	31.19 ± 0.70	6.40 ± 0.18	0.760 ± 0.0004	0.726 ± 0.002	0
Fine-tuned	Llama-3.2-3B	30.51 ± 0.76	5.81 ± 0.10	0.755 ± 0.001	0.779 ± 0.001	0
Zero-shot, Alpaca	Llama-3.1-8B	9.89	1.41	0.500 (0.583)	2.027	141
Zero-shot, Alpaca	Llama-3.2-3B	12.51	1.01	0.587	2.225	4
Zero-shot, bare	Llama-3.1-8B	13.42	0.40	0.533 (0.541)	2.082	14
Zero-shot, bare	Llama-3.2-3B	12.65	0.38	0.536	2.276	9

6 Discussion

6.1 What Likelihood and Generation Overlap Each Show

Across the four adapted models, BPB separated every pair. The overlap metrics separated every pair except Llama-3.2-3B and Qwen2.5-3B, which had the largest likelihood gap among the small models. The two kinds of measure answer different questions. BPB measures how probable a model finds one human-written response under teacher forcing; chrF++ and BLEU measure how close the model's own greedy output is to that response.

Several mechanisms could produce the gap, and our design does not test them. Teacher-forced scoring conditions each token on the reference prefix, whereas free-running greedy decoding conditions on the model's own earlier choices, so a model can rank the reference well without producing it. With one reference per item, many adequate responses score poorly on overlap, which adds noise that likelihood does not have. Greedy decoding with a 256-token limit and a repetition penalty changes the generated text in ways that likelihood scoring does not see, and the 9–12% of references that are longer than the limit cannot be matched in full by any model; output length itself, however, was similar across models (Section 5.5). Distillation could also play a part: Llama-3.2-3B was pretrained with token-level targets from larger Llama models [30]. Gemma-2-2B was also distilled [12] yet has the highest BPB, so distillation alone does not explain the pattern.

Meister and Cotterell [28] argued that perplexity does not characterize the text a model generates; with byte normalization, the dissociation holds even though likelihood is compared across different tokenizers. The practical consequence is that neither kind of measure can stand in for the other: a study that selected among our models by likelihood would have preferred Llama-3.2-3B over Qwen2.5-3B, and one that selected by chrF++ would have found no difference. The evidence comes from four models and one test set.

6.2 What Tokenizer Fertility Can and Cannot Indicate

Fertility ranked the tokenizers consistently, on the benchmark references and under three romanization conventions, but it did not identify the best model: the tokenizer that split words least belonged to the model that scored lowest. Three points limit what this shows. The range among the trained models was narrow (2.72–3.00 tokens per word), far from the many-fold differences between languages [34] or between native and romanized scripts [17] for which fertility is known to matter. The models differ in much more than the tokenizer (Table 2). And fertility itself depended on the romanization convention about as much as on the tokenizer (Section 5.1).

The result agrees with controlled pretraining studies in which fertility or token count did not reliably predict downstream performance [1, 43], and with transfer experiments in which extending the vocabulary to shorten target-language text was not needed [50]. It does not contradict the finding that compression tracks performance when everything except the tokenizer is held fixed [13]. Our measurements also do not support explaining Llama-family results by over-fragmentation [41]: on the same text, the Llama 3 tokenizer split words into fewer pieces than the Qwen and Mistral tokenizers.

One mechanism deserves attention in future work. A large vocabulary lowers fertility but costs embedding parameters. Gemma-2-2B devotes 22.6% of its parameters to the embeddings of its 256k vocabulary and has the fewest non-embedding parameters of the three models (2.02B against 2.77–2.82B), so in our models lower fertility and lower non-embedding capacity come together, and the data cannot separate them.

6.3 Interpreting Gemma-2-2B

Gemma-2-2B is not an adaptation failure. Its chrF++ gain under the Alpaca prompt (+18.89) is the largest of the three small models, and its BPB falls by the same proportion as the others' (to 0.349 of its zero-shot value). It starts lowest and ends lowest. Several differences could contribute, and our design cannot separate them: the fewest non-embedding parameters and pretraining tokens of the three models (Table 2); pretraining data that are primarily English [12]; attention and final-logit soft-capping; Unsloth compilation disabled; a recipe tuned for none of the models; and a late rise in validation loss to 1.14–1.15 times its best earlier value. Its zero-shot behaviour, with the most English-like output across the two prompts and the lowest zero-shot chrF++ of the three small models, fits less pretraining exposure to Romanized Nepali, but exposure was not measured.

6.4 Zero-Shot Baselines and Gains

On this task, zero-shot baselines measure language choice and prompt-following as much as ability. Depending on the model and prompt, the base models answered in English, copied the prompt, looped or returned nothing. Gains over these baselines therefore mix a switch of output language with task learning, and the order of models by gain changed with the prompt and with the metric (Table 8). Rimal and Rimal [41] read such gains as evidence that models with weaker zero-shot performance have more representational plasticity. Our data do not support ranking models by gains when the base models mostly answer in another language: the model with the largest chrF++ gain under one prompt had the smallest under the other. The proportional fall in BPB was nearly constant across the four models and both prompts (64–66%), which suggests that likelihood is a steadier summary of how much a model changes with adaptation, within the limits discussed above. The concentration of zero-shot failures on type B prompts is consistent with base models having difficulty reading Romanized Nepali instructions.

6.5 The Within-Family Comparison

With the tokenizer held fixed, the larger Llama model had clearly higher reference likelihood before and after fine-tuning, but after fine-tuning its outputs were only slightly closer to the references: by 0.68 chrF++ and 0.58 BLEU points, gains that hold across test items but, for chrF++, not across seeds. The comparison thus repeats, within one family and without any difference in segmentation, the pattern seen across families: a likelihood advantage that carries over to generation only in part. It cannot isolate the effect of parameter count, because Llama-3.1-8B and Llama-3.2-3B differ in model generation, data and distillation as well as size; Llama-3.2-3B was trained in part on the larger models' output distributions [30].

6.6 Implications

Four practical points follow for adapting models to romanized varieties such as Romanized Nepali. First, tokenizer fertility should not be used on its own to choose among pretrained models; within the narrow range of modern tokenizers, other differences dominated. Second, the romanization convention is a design variable. In our measurements, lowercasing changed fertility by 13–14% and dropping the final inherent vowel by a further 3–4%, and the convention decides which distinctions of the script a model can recover, so benchmarks should report it. Third, adapted models should be evaluated with a byte-normalized likelihood together with generation metrics and an output audit, because the two kinds of measure separate models differently. Fourth, comparisons need several seeds and paired tests: chrF++ differences of 1.5–1.6 points were resolved by paired item-level tests but not across three seeds, and neither kind of interval alone captures both sources of variation.

7 Limitations

Data. All results come from one instruction dataset of 10,500 examples, derived from a Nepali translation of the Stanford Alpaca data whose dataset card documents neither its provenance nor a licence. Romanized text was produced by deterministic, lowercased ITRANS transliteration. This scheme keeps word-final inherent vowels, merges 11 groups of letters and gives every word one spelling, so the benchmark does not test robustness to the spelling variation of naturally typed Romanized Nepali, to mixed scripts or to code-switching. Type A instructions were machine-translated back into English from a Nepali translation of English originals, which sometimes changes their meaning, and the two types differ in how the instruction was produced as well as in its language. Some training examples (1.4–2.0%) were truncated at 512 tokens, and a few outputs recur across splits.

Models and recipe. Three base-model families were trained with one shared recipe that was not tuned per model. The models differ in tokenizer, architecture, non-embedding parameter count, pretraining data and distillation at once, so no

difference between them can be attributed to any single factor, including the tokenizer. Unsloth left different modules unquantized in different models.

Evaluation. All measures are automatic and single-reference. BLEU, chrF++ and ROUGE reward overlap with one reference; BERTScore relies on a multilingual encoder that has not been validated for Romanized Nepali; and bits per byte measures how probable the reference is, not how good a generated response is. The output audit uses surface heuristics, and its English-stopword list was screened against the test references. Zero-shot results depend on two prompt formats and our decoding settings.

Statistics. Three seeds give only a rough estimate of training variability. The paired item-level intervals condition on the trained seeds and the seed-level intervals on the test set; neither captures both sources of variation at once. BERTScore could not be recomputed from the saved outputs, so its values are those of the evaluation scripts. The divergence criterion was set post hoc, and the selected checkpoint steps were not retained.

Fertility analysis. The public-sample analysis used tokenizer files rebuilt from public sources, which reproduce the Hugging Face token counts exactly on the benchmark references, and a simple heuristic for dropping the final inherent vowel. Its sentences have shorter words than the benchmark responses.

Within-family comparison. Llama-3.1-8B and Llama-3.2-3B differ in model generation, training data and distillation as well as size. The comparison does not measure an effect of parameter count.

Software. Package versions differed across runs. One re-evaluation in a second environment reproduced its results exactly, but it covers one adapter.

Independent human assessment and evaluation on naturally written Romanized Nepali prompts are the main directions for future work.

8 Conclusion

We asked how small open language models adapt to Romanized Nepali and whether common shortcuts identify the best one. All three 2–3B families adapted substantially under a shared QLoRA recipe, and their results varied across seeds by about as much as across test samples (RQ1). The shortcuts did not hold. The tokenizer that fragmented Romanized Nepali least belonged to the lowest-scoring model (RQ2). The model that assigned the highest likelihood to the references did not generate text closer to them than its nearest competitor, and within the Llama family, where tokenization was identical, the larger model's likelihood advantage carried over to generation only in part (RQ3). Without adaptation the base models mostly answered in English, and which model gained most from fine-tuning depended on the zero-shot prompt (RQ4). For low-resource and romanized varieties, where a single proxy is tempting, these results argue for evaluating adapted models with several kinds of measure and explicit uncertainty. These conclusions hold for the transliterated benchmark on which they were obtained.

Ethics Statement

We used a publicly available instruction dataset whose card states no licence. It is a Nepali translation of the Stanford Alpaca data (Section 3.1), whose CC BY-NC 4.0 licence restricts it to non-commercial use. We collected no personal data and recruited no participants. The sentences used in the fertility analysis come from the Mozilla Common Voice text corpus, released under CC0, and a Nepali translation of the Universal Declaration of Human Rights distributed with NLTK. Models were used under their licences: the Llama 3.2 and Llama 3.1 Community Licenses, the Gemma Terms of Use and the Qwen Research License, which restricts Qwen2.5-3B to research use. Fine-tuned models can produce incorrect or biased text, and even the fine-tuned models occasionally copied prompts, looped or switched to English, so they should not be used in high-stakes settings without human review. The transliteration scheme determines which spellings the models learn and does not reflect how all writers of Romanized Nepali spell.

Data and Code Availability

The training and evaluation scripts, the code and results of the analyses in this paper, and the per-item generations, reference NLL values and token and byte counts of the 19 evaluation runs whose outputs were retained are available at https://github.com/pantpujan017/romanized-nepali-slm-adaptation. Because the source dataset states no licence, we release model outputs rather than its text.

Acknowledgments

During the preparation of this study, the authors used Claude Opus 5.5 (Anthropic) to draft and refine the manuscript text, to check references, to write the code for the re-analysis of the saved outputs (Tables 8 and 9 and Appendix C) and for the tokenizer and romanization analysis (Section 5.1 and Appendix B), and to prepare the plotting code for Figures 1 and 2. Claude (Anthropic) was also used to write the code for the experiments, including the training and evaluation scripts. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

References

[1] Mehdi Ali, Michael Fromm, Klaudia Thellmann, et al.. 2024. Tokenizer Choice For LLM Training: Negligible or Crucial?. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, Mexico City, Mexico, 3907–3924. https://doi.org/10.18653/v1/2024.findings-naacl.247

[2] Rosana Ardila, Megan Branson, Kelly Davis, et al.. 2020. Common Voice: A Massively-Multilingual Speech Corpus. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC 2020)*. European Language Resources Association, Marseille, France, 4218–4222. https://aclanthology.org/2020.lrec-1.520

[3] H. Baruah, S. R. Singh, and P. Sarmah. 2024. Transliteration Characteristics in Romanized Assamese Language Social Media Text and Machine Transliteration. *ACM Transactions on Asian and Low-Resource Language Information Processing* 23, 2, Article 33 (2024). https://doi.org/10.1145/3639565

[4] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media, Sebastopol, CA.

[5] C. P. Chaudhary, B. Joshi, A. Shakya, and S. Giri. 2025. NEPTUN: Normalization for Romanized Nepali Sentiment Analysis. *Journal of Himalaya College of Engineering* 2, 1 (2025), 19–28. https://doi.org/10.3126/jhcoe.v2i1.91508

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 10088–10115.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Minneapolis, MN, 4171–4186. https://doi.org/10.18653/v1/N19-1423

[8] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. 2020. Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping. arXiv:2002.06305. https://arxiv.org/abs/2002.06305

[9] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Melbourne, Australia, 1383–1392. https://doi.org/10.18653/v1/P18-1128

[10] S. Duwal, S. Prasai, and S. Manandhar. 2025. Domain-adaptative Continual Learning for Low-resource Tasks: Evaluation on Nepali. In *Proceedings of the First Workshop on Challenges in Processing South Asian Languages (CHiPSAL 2025)*. Association for Computational Linguistics, Abu Dhabi, UAE, 144–153. https://aclanthology.org/2025.chipsal-1.14

[11] Leo Gao, Stella Biderman, Sid Black, et al.. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. arXiv:2101.00027. https://arxiv.org/abs/2101.00027

[12] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. https://arxiv.org/abs/2408.00118

[13] Omer Goldman, Avi Caciularu, Matan Eyal, Kris Cao, Idan Szpektor, and Reut Tsarfaty. 2024. Unpacking Tokenization: Evaluating Text Compression and its Correlation with Model Performance. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 2274–2286. https://doi.org/10.18653/v1/2024.findings-acl.134

[14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al.. 2024. The Llama 3 Herd of Models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783

[15] Daniel Han, Michael Han, and Unsloth team. 2023. *Unsloth*. GitHub repository. https://github.com/unslothai/unsloth

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the Tenth International Conference on Learning Representations (ICLR 2022)*. https://openreview.net/forum?id=nZeVKeeFYf9

[17] Jaavid Aktar Husain, Raj Dabre, Aswanth Kumar, Jay Gala, Thanmay Jayakumar, Ratish Puduppully, and Anoop Kunchukuttan. 2024. RomanSetu: Efficiently Unlocking Multilingual Capabilities of Large Language Models via Romanization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 15593–15615. https://doi.org/10.18653/v1/2024.acl-long.833

[18] Indic Transliteration Project. 2026. *indic_transliteration: Transliteration Tools to Convert Text between Indic Scripts and Romanization Schemes (version 2.3.82)*. Python package, MIT License. https://github.com/indic-transliteration/indic_transliteration_py

[19] Akif Islam and Mohd Ruhul Ameen. 2025. Parameter-Efficient Fine-Tuning for Low-Resource Languages: A Comparative Study of LLMs for Bengali Hate Speech Detection. In *Proceedings of IEEE COMPAS 2025*. IEEE. https://doi.org/10.1109/COMPAS67506.2025.11381789

[20] Damjan Kalajdzievski. 2023. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. arXiv:2312.03732. https://arxiv.org/abs/2312.03732

[21] O. Khade, S. Jagdale, A. Phaltankar, G. Takalikar, and R. Joshi. 2025. Challenges in Adapting Multilingual LLMs to Low-Resource Languages using LoRA PEFT Tuning. In *Proceedings of the First Workshop on Challenges in Processing South Asian Languages (CHiPSAL 2025)*. Association for Computational Linguistics, Abu Dhabi, UAE, 217–222. https://aclanthology.org/2025.chipsal-1.22

[22] Rajesh Khatiwada. 2009. Nepali. *Journal of the International Phonetic Association* 39, 3 (2009), 373–380. https://doi.org/10.1017/S0025100309990181

[23] M. Khullar, U. Desai, P. Malviya, A. Dalmia, and Z. R. Shi. 2025. Script Gap: Evaluating LLM Triage on Indian Languages in Native vs Roman Scripts in a Real World Setting. arXiv:2512.10780. https://arxiv.org/abs/2512.10780

[24] Philipp Koehn. 2004. Statistical Significance Tests for Machine Translation Evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Barcelona, Spain, 388–395. https://aclanthology.org/W04-3250

[25] Taku Kudo and John Richardson. 2018. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Brussels, Belgium, 66–71. https://doi.org/10.18653/v1/D18-2012

[26] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. https://aclanthology.org/W04-1013

[27] N. Luitel, N. Bekoju, A. K. Sah, and S. Shakya. 2025. Can Perplexity Predict Finetuning Performance? An Investigation of Tokenization Effects on Sequential Language Models for Nepali. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM^2)*. Association for Computational Linguistics, 239–248. https://aclanthology.org/2025.gem-1.21

[28] Clara Meister and Ryan Cotterell. 2021. Language Model Evaluation Beyond Perplexity. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 5328–5339. https://doi.org/10.18653/v1/2021.acl-long.414

[29] Meta. 2024a. *Llama 3.1 Model Card*. Retrieved November 9, 2026 from https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md

[30] Meta. 2024b. *Llama 3.2 Model Card*. Retrieved November 9, 2026 from https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md

[31] Nobal B. Niraula, Saurab Dulal, and Diwa Koirala. 2021. Offensive Language Detection in Nepali Social Media. In *Proceedings of the 5th Workshop on Online Abuse and Harms (WOAH 2021)*. Association for Computational Linguistics, Online, 67–75. https://doi.org/10.18653/v1/2021.woah-1.7

[32] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Philadelphia, PA, 311–318. https://doi.org/10.3115/1073083.1073135

[33] S. Patel, K. K. Dhami, N. Sherpa, and S. Khadka. 2026. From Romanized to Devanagari: Enhancing Nepali Sentiment Analysis with NepaliXlit. In *Proceedings of the Second Workshop on Challenges in Processing South Asian Languages (CHiPSAL 2026)*. 115–126. https://doi.org/10.63317/5j8kkeb2myf6

[34] Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. Language Model Tokenizers Introduce Unfairness Between Languages. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract.html

[35] Maja Popović. 2015. chrF: Character n-gram F-score for Automatic MT Evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*. Association for Computational Linguistics, Lisbon, Portugal, 392–395. https://doi.org/10.18653/v1/W15-3049

[36] Maja Popović. 2017. chrF++: Words Helping Character n-grams. In *Proceedings of the Second Conference on Machine Translation*. Association for Computational Linguistics, Copenhagen, Denmark, 612–618. https://doi.org/10.18653/v1/W17-4770

[37] Matt Post. 2018. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*. Association for Computational Linguistics, Brussels, Belgium, 186–191. https://doi.org/10.18653/v1/W18-6319

[38] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115. https://arxiv.org/abs/2412.15115

[39] P. Rai, S. Chatterji, and B.-G. Kim. 2023. Deep Learning-Based Sequence Labeling Tools for Nepali. *ACM Transactions on Asian and Low-Resource Language Information Processing* 22, 8, Article 211 (2023). https://doi.org/10.1145/3606696

[40] M. Rajapakse and R. Weerasinghe. 2026. A Comprehensive Benchmark of Language Models on Unicode and Romanized Sinhala. arXiv:2601.14958v1. https://arxiv.org/abs/2601.14958v1

[41] Ananda Rimal and Adarsha Rimal. 2026. Benchmarking Linguistic Adaptation in Comparable-Sized LLMs: A Study of Llama-3.1-8B, Mistral-7B-v0.1, and Qwen3-8B on Romanized Nepali. arXiv:2604.14171. https://arxiv.org/abs/2604.14171

[42] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 3118–3135. https://doi.org/10.18653/v1/2021.acl-long.243

[43] Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. 2024. Tokenization Is More Than Compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, FL, 678–702. https://doi.org/10.18653/v1/2024.emnlp-main.40

[44] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 1715–1725. https://doi.org/10.18653/v1/P16-1162

[45] J. Sha, M. Zhu, C. Feng, and J. Ci. 2025. Tibetan-LLaMA 2: Large Language Model for Tibetan. *ACM Transactions on Asian and Low-Resource Language Information Processing* 24, 12, Article 141 (2025). https://doi.org/10.1145/3776748

[46] M. Sharma, S. Khadka, U. C. Aryal, et al.. 2026. Evaluating Large Language Models' Responses to Sexual and Reproductive Health Queries in Nepali. arXiv:2603.22291. https://arxiv.org/abs/2603.22291

[47] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. *Stanford Alpaca: An Instruction-following LLaMA Model*. GitHub repository. https://github.com/tatsu-lab/stanford_alpaca

[48] Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozcelik. 2023. Impact of Tokenization on Language Models: An Analysis for Turkish. *ACM Transactions on Asian and Low-Resource Language Information Processing* 22, 4, Article 116 (2023). https://doi.org/10.1145/3578707

[49] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *Proceedings of the Eighth International Conference on Learning Representations (ICLR 2020)*. https://openreview.net/forum?id=SkeHuCVFDr

[50] Jun Zhao, Zhihao Zhang, Luhui Gao, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. LLaMA Beyond English: An Empirical Study on Language Capability Transfer. arXiv:2401.01055. https://arxiv.org/abs/2401.01055

A Run Provenance

Hashes are SHA-256 prefixes; full hashes are in the code repository. "Not recorded" marks information for which no run artifact was retained.

Table 11: Scripts, environments and records behind the reported numbers.

Item	Detail
Dataset splits	train 9ed86da7…, validation 3f7f6854…, test d656353e…; hash-identical copies of the dataset directory were uploaded to Kaggle more than once and read by different sessions and accounts (the zero-shot launch records and the 8B seed-44 run use a second copy); all 19 retained runs have the same 1,000 test references.
Tokenizer revisions	Fertility and length analyses (Transformers 5.16.1, tokenizers 0.23.1): Llama-3.2-3B 13afe512…, Qwen2.5-3B 3aab1f19…, gemma-2-2b c5ebcd40…, Llama-3.1-8B d04e592b…, Mistral-7B-v0.1 27d67f1b…, Qwen3-8B b968826d…, Qwen3-8B-Base 49e3418f…; fertility script a0c856c5…, length-audit script 3938c8a4….
Model weights	Loaded through Unsloth, which substitutes its pre-quantized 4-bit checkpoints (Section 4.2); their revisions were not recorded.
Training, primary models	train.py v2 (579d36c7…, canonical) with per-run patched copies; Qwen2.5-3B seeds 43 and 44 were trained in parallel, one per GPU, with Unsloth 2026.10.11, from a copy that differs only in its CONFIG values and one comment line (d9d03919…); the excluded second seed-42 replicate used train.py v1, which has the same training settings, with Unsloth 2026.10.11 and Transformers 5.16.1; other Unsloth versions, bitsandbytes and TRL versions not recorded.
Training, Llama-3.1-8B	canonical train.py with only the model, data and output paths changed; patched copies b6949e2f… (seed 42), d6212c1f… (seed 43) and 662c1093… (seed 44); Unsloth 2026.11.3; seed 44 trained in a separate Kaggle account from a hash-identical copy of the data.
Fine-tuned evaluation, primary models	eval.py v2.2 (metric files record the script hash as unknown); Llama-3.2-3B evaluations used Unsloth 2026.10.11 (seed 42) and 2026.11.1 (seeds 43 and 44) with Transformers 5.17.0, PEFT 0.21.2 and torch 2.11.0+cu130; Qwen2.5-3B seed 42 was evaluated with Unsloth 2026.10.11 and Transformers 5.16.1, and seeds 43 and 44 with a copy of eval.py v2.2 whose hash was recorded (a231a1ae…); other Qwen2.5-3B and the Gemma-2-2B evaluation versions not recorded.
Zero-shot evaluation, primary models	one script (33e10eac…) with per-run patched copies. Qwen2.5-3B Alpaca: patched-copy hash not recorded, Unsloth 2026.11.2. Qwen2.5-3B bare: 618c89fb…, started 2026-11-06 02:39:01 UTC, Unsloth 2026.11.2. Llama-3.2-3B Alpaca: 11191d0c…, started 2026-11-07 01:25:20 UTC, Unsloth 2026.11.3. Llama-3.2-3B bare: c8a28578…, Unsloth 2026.11.2; launch records give 2026-11-06 08:02:48 UTC (recovery record) and 12:51:30 UTC (launch file). Both Gemma-2-2B runs: Unsloth 2026.11.3.
Metric recovery	recover_metrics.py (ba4a20c5…), used for zero-shot runs with empty generations, which keeps empty hypotheses at BERTScore 0; validated on the Qwen2.5-3B Alpaca run (Section 4.6).
Llama-3.1-8B evaluation	eval.py v2.4 (51e1bd41…); patched copies f34da070… (seed 42), 12c1f7c3… (seed 43) and c288f295… (seed 44); Unsloth 2026.11.3, Transformers 5.17.0, PEFT 0.21.2 and torch 2.11.0+cu130; launch times not recorded.
Llama-3.1-8B zero-shot	Alpaca: patched copy a587b947…, started 2026-11-08 04:50:24 UTC; bare: generation script 7f36ac2f…; metrics rebuilt with recover_metrics.py. The bare-prompt outputs were not retained.
Output audit	output_audit.py (69b88899…), applied to all 20 evaluation runs and the reference responses.
Re-evaluation check	Qwen2.5-3B seed 43 re-evaluated with Unsloth 2026.11.2, Transformers 5.17.0, PEFT 0.21.2 and PyTorch for CUDA 13.0: identical generations, per-item NLL values and metrics.
Metric signatures	BLEU `nrefs:1|case:mixed|eff:no|tok:13a|smooth:exp|version:2.6.0`; chrF++ `nrefs:1|case:mixed|eff:yes|nc:6|nw:2|space:no|version:2.6.0`; evaluation-script intervals used 200 resamples with Python random seed 0, re-initialized for each subset.
Re-analysis	run_level_analysis.py in the code repository (sacreBLEU 2.6.0, rouge-score 0.1.2), applied to the per-item outputs of 19 runs; it reproduces every reported chrF++, BLEU, ROUGE and BPB value.

Table 12: Training runs and their use. Checkpoints were selected on validation loss (Section 4.3); the retained records give the selected step only where noted.

Model	Seed	Status	Unsloth (training)	Notes
Qwen2.5-3B	42	used (first run)	not recorded	
Qwen2.5-3B	42	excluded (diverged)	2026.10.11	second replicate, train.py v1
Qwen2.5-3B	43	used	2026.10.11	selected step 2200
Qwen2.5-3B	44	used	2026.10.11	selected step 2200
Llama-3.2-3B	42	used	not recorded	
Llama-3.2-3B	43	used	not recorded	one non-finite gradient norm (step 2150)
Llama-3.2-3B	44	used	not recorded	
Gemma-2-2B	42	used	not recorded	compilation disabled
Gemma-2-2B	43	used	not recorded	compilation disabled
Gemma-2-2B	44	used	not recorded	compilation disabled
Llama-3.1-8B	42	used	2026.11.3	
Llama-3.1-8B	43	used	2026.11.3	
Llama-3.1-8B	44	used	2026.11.3	separate Kaggle account

B Romanization and Fertility Analysis

Corpus. The public sample combines 1,146 sentences from the Mozilla Common Voice Nepali text corpus (CC0) [2] and 87 lines of the Nepali Universal Declaration of Human Rights from NLTK's udhr2 corpus [4], after Unicode NFC normalization and removal of duplicates and lines without Devanagari. It has 8,840 whitespace-delimited words; the 8,102 that remain after removing standalone punctuation are used for the lexical statistics in Section 3.2.

Conventions. Devanagari was converted to ITRANS with indic_transliteration 2.3.82 [18] (R1), then lowercased (R2). R3 removes the final *a* from R2 words whose Devanagari form ends in a consonant letter without a virama and has at least two aksharas; words ending in a candrabindu, anusvara or visarga are unchanged. Trailing punctuation is split off before transliteration, so the danda becomes "।". Under R2, the 11 groups in Table 13 merge; lowercasing also merges the anusvara with a virama-marked म (both *m*) and the visarga with a virama-marked ह (both *h*), and it merges the nukta letters ग़, ख़, ळ and ऱ with their plain forms, although these are rare in Nepali.

Table 13: Letter groups merged by lowercasing ITRANS. Vowel groups include the corresponding vowel signs.

Devanagari	ITRANS (case-preserving)	Lowercased
च / छ	`ch / Ch`	`ch`
त / ट	`t / T`	`t`
थ / ठ	`th / Th`	`th`
द / ड	`d / D`	`d`
ध / ढ	`dh / Dh`	`dh`
न / ण	`n / N`	`n`
श / ष	`sh / Sh`	`sh`
ञ / ङ	`~n / ~N`	`~n`
अ / आ	`a / A`	`a`
इ / ई	`i / I`	`i`
उ / ऊ	`u / U`	`u`

Tokenizers. Hugging Face model files could not be retrieved in the analysis environment, so tokenizers were rebuilt from public files with each model's published pre-tokenization: the Llama 3 BPE ranks from Meta's llama-models repository with the Llama 3 split pattern, the Qwen BPE ranks (151,643 tokens) distributed with the dashscope 2.0.4 package with Qwen's pattern (single digits), the Gemma SentencePiece model from the gemma_pytorch repository, and the Mistral v0.1 SentencePiece model from mistral-common 1.13.1. Word-level counts encode each word with a leading space, or with SentencePiece's own prefix for Mistral. As a check, these tokenizers reproduce the Hugging

Face token counts of the Llama-3.2-3B, Qwen2.5-3B, gemma-2-2b and Mistral-7B-v0.1 tokenizers exactly on all 1,000 benchmark references. Fertility is the mean over sentences of tokens per word, with 1,000 bootstrap resamples over sentences (seed 0); the order of tokenizers was checked over 2,000 paired resamples (seed 1). The script and its full output, including intervals for all conventions and type-level fertility, are in the code repository.

C Per-Seed Results

Table 14 gives every fine-tuned run, recomputed from the saved outputs. The values reproduce those of the evaluation scripts, and the means and SDs of Tables 7 and 10 follow from them.

Table 14: Per-seed results of the fine-tuned runs on the 1,000 test items. Length is total generated characters divided by total reference characters.

Model	Seed	chrF++	BLEU	ROUGE-L	BPB	chrF++ type A	chrF++ type B	Length
Qwen2.5-3B	42	29.53	5.61	0.202	0.8208	30.70	28.39	0.75
Qwen2.5-3B	43	31.42	5.84	0.201	0.8243	32.68	30.21	0.84
Qwen2.5-3B	44	30.87	6.13	0.205	0.8182	32.16	29.61	0.79
Llama-3.2-3B	42	30.19	5.82	0.201	0.7795	31.67	28.75	0.79
Llama-3.2-3B	43	31.38	5.91	0.200	0.7779	32.64	30.15	0.83
Llama-3.2-3B	44	29.97	5.72	0.207	0.7790	31.45	28.53	0.76
Gemma-2-2B	42	28.68	4.51	0.175	0.8432	29.86	27.55	0.78
Gemma-2-2B	43	28.64	4.99	0.180	0.8424	29.89	27.43	0.76
Gemma-2-2B	44	29.73	4.84	0.172	0.8393	30.99	28.51	0.81
Llama-3.1-8B	42	30.79	6.20	0.211	0.7263	32.47	29.15	0.76
Llama-3.1-8B	43	30.78	6.46	0.208	0.7274	32.00	29.60	0.76
Llama-3.1-8B	44	32.00	6.54	0.212	0.7236	33.30	30.74	0.82